

2026-05-06-p1-f19-ask-user-question

P1-F19 — AskUserQuestion with Previews Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development. Steps use checkbox (- []) syntax for tracking.

Goal: Ship a real, end-to-end `ask_user` tool for the HelixCode CLI agent. F19 adds an `internal/tools/askuser/` package with `Choice / Question / Result` value types, a `Prompter` interface, a production `stdinPrompter` (real `stdin` readline + retry-on-invalid + timeout + non-TTY graceful fallback), and an `AskUserTool` (`tools.Tool` impl) that the registry exposes under the stable name `ask_user` (snake_case; Q3=A) in the new category `tools.CategoryAskUser`. `Stdin` readline reuses F18's `render.RenderLines` to format the question + per-choice inline preview (Q2=A) + numbered menu; the user types a number + Enter; ENTER alone with `default` set picks the default. Non-TTY (`term.IsTerminal=false`; Q4=B) short-circuits BEFORE any read: with `default` set it returns the default with `Source="default"`; without default it returns `ErrNoTTYNoDefault` ("ask_user requires interactive terminal AND no default specified"). **Tool only** (Q5=A) — NO slash command, NO cobra subcommand. **No new external dependencies** — pure `stdlib` (`bufio`, `context`, `errors`, `fmt`, `io`, `os`, `strconv`, `strings`, `sync`, `time`) + `golang.org/x/term` (already a direct dep after F18) + `dev.helix.code/internal/render`.

Architecture: New `internal/tools/askuser/` package with `types.go` (`Choice + Question + Result + Prompter` interface + sentinels `ErrInvalidQuestion/ErrNoTTYNoDefault/ErrUserCancelled/ErrPromptTimeout/ErrTooManyInvalidAttempts` + constants `DefaultTimeout=5min/DefaultMaxRetries=3/SourceStdin/SourceDefault`), `stdin_prompter.go` (production `stdinPrompter` with `Reader/Writer/IsTTY/Renderer/Timeout/MaxRetries` constructor seams; `FormatQuestion` exported pure helper; non-TTY branch short-circuits before read; TTY branch reads via `bufio.ReadString('\n')` in a goroutine with `select-on-ctx.Done` for cancellation; retries up to `MaxRetries` on empty/non-numeric/out-of-range; EOF → `ErrUserCancelled`; `ctx-cancel/timeout` → `ErrPromptTimeout`), `ask_user_tool.go` (`AskUserTool` implements `tools.Tool`; `Name()`=="ask_user"; `Category()`==`tools.CategoryAskUser`; parses `params["question"]/params["choices"]/params["default"]`; calls `prompter.Prompt`; returns `Result` as `map[string]interface{}`). Two existing files get tiny additions: `internal/tools/registry.go` (new `CategoryAskUser` `ToolCategory = "ask-user" const` + optional `AskUserPrompter` `askuser.Prompter` field on `RegistryConfig` + registration in `buildToolList`); `cmd/cli/main.go` (no changes required for correctness; optional polish to share `c.renderer` via `RegistryConfig.AskUserPrompter`).

Tech Stack: Go 1.26, testify v1.11, zap (already in `go.mod`) — already present. **NO new external deps.** `golang.org/x/term` is a direct dep at v0.41.0 (promoted from indirect by F18-T06; verified via `grep "golang.org/x/term" helix_code/go.mod`). `dev.helix.code/internal/render` is the F18-internal package; `internal/tools/askuser/` imports `render` for `RenderLines` only. `go mod tidy` after T02 must produce

zero new entries in go.sum AND zero changes to go.mod. T07's verification step asserts this loudly.

Spec: docs/superpowers/specs/2026-05-06-p1-f19-ask-user-question-design.md (commit cedd81e)

Working directory for go commands: helix_code/. Git from meta-repo root.

Anti-bluff smoke (FULL 4-term applied to F19 surface):

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
placeholder" \
    internal/tools/askuser && echo BLUFF || echo clean
```

Must always print clean.

Anti-bluff hot zone: §5.2 of the spec — F19 can degenerate in four ways: (a) Prompt short-circuits to the default without checking IsTTY (so any test with default gets the default, hiding stdin-read regressions); (b) the non-TTY error path is never tested (a regression that *blocks* on os.Stdin in non-TTY mode is silently shipped); (c) default-pick behavior is tested only with mocks (the production stdinPrompter is never exercised; a real-world non-TTY invocation hangs); (d) preview text is rendered AFTER the choice label (or omitted), defeating the entire point. The four “what counts as ask_user works” criteria — (1) production stdinPrompter reads from the injected reader and consumes the expected bytes; (2) non-TTY-without-default returns ErrNoTTYNoDefault with ZERO bytes consumed (reader-position invariant); (3) non-TTY-with-default returns the default with ZERO bytes consumed AND Source="default"; (4) preview text appears in the captured prompt output BEFORE the choice label (byte-offset invariant) — are each tested with both unit assertions AND a Challenge phase. The Challenge harness uses positive evidence: returned Result.Value/Source/Index, captured byte content, byte offsets, and reader-position invariants. Disk-state-equivalent (reader-position) mismatch is a hard Challenge failure. Absence-of-error is NEVER acceptable.

Why this is consequential: the agent asking the user a structured question is the difference between “the agent guesses and apologises later” and “the agent confirms the destructive operation before running it”. F19's discriminating tests are: (i) the Challenge's PHASE-B (non-TTY + default + reader has "NEVER\n"; assert reader bytes unchanged AND Source="default" — proves the production class short-circuits BEFORE reading); (ii) the Challenge's PHASE-C (non-TTY + no default; assert errors.Is(err, ErrNoTTYNoDefault) AND reader bytes unchanged — proves the production class errors loudly instead of blocking); (iii) the Challenge's PHASE-D (TTY + previews; capture writer; assert byte offsets put each preview BEFORE its choice label — proves the menu is rendered correctly, not just “rendered”); (iv) the Challenge's PHASE-E (TTY + reader "9\n1\n"; assert prompter consumes both lines AND retry message appears exactly once — proves the retry loop reads new bytes, not the same ones in a loop). All four must produce positive evidence; none can be satisfied by absence-of-error.

Task list



P1-F19-T01 — bootstrap evidence + advance PROGRESS to F19



P1-F19-T02 — internal/tools/askuser/types.go: Choice + Question + Result + Prompter interface + sentinels + constants (TDD)



P1-F19-T03 — `internal/tools/askuser/stdin_prompter.go`: `stdinPrompter` using F18 renderer + non-TTY short-circuit + retry loop + timeout (TDD against `bytes.Buffer`)



P1-F19-T04 — `internal/tools/askuser/ask_user_tool.go`: `AskUserTool` wrapping `Prompter` + add `CategoryAskUser` to registry (TDD)



P1-F19-T05 — `cmd/cli/main.go` wiring (register tool through registry config) + integration test (always-runs both branches via real `bytes.Buffer`)



P1-F19-T06 — Challenge harness (5 always-run phases: STDIN-INPUT + NON-TTY-WITH-DEFAULT + NON-TTY-WITHOUT-DEFAULT-ERR + PREVIEW-RENDERING + INVALID-INPUT-RETRY) with positive byte/reader-position evidence



P1-F19-T07 — Feature 19 close-out + push 4 remotes non-force

Task 1: Bootstrap

Append F19 evidence section header (spec `cedd81e`), update PROGRESS current focus to F19 (replacing the F18 close-out's "F19 next candidate" pointer), insert F19 task list (7 items) after F18's. Confirm `06_phase_1_evidence.md` has an F19 anchor. Verify `golang.org/x/term v0.41.0` is a direct dep in `helix_code/go.mod` (sanity check before T03 imports it).

Commit: `docs(P1-F19-T01): bootstrap Phase 1 / Feature 19 evidence + advance PROGRESS.`

Task 2: types.go (TDD)

Files: new `helix_code/internal/tools/askuser/types.go`, new `helix_code/internal/tools/askuser/types_test.go`.

Define: `- Choice{Label, Value, Preview string}. - Question{Question string; Choices []Choice; Default string} + Validate() error method. - Result{Value, Label string; Index int; Source string}. - Prompter interface(Prompt(ctx, q Question) (Result, error)). - Error sentinels (ErrInvalidQuestion, ErrNoTTYNoDefault, ErrUserCancelled, ErrPromptTimeout, ErrTooManyInvalidAttempts). - Constants (DefaultTimeout = 5 * time.Minute, DefaultMaxRetries = 3, SourceStdin = "stdin", SourceDefault = "default").`

Failing tests FIRST:

```
func TestChoice_FieldsZeroValueOK(t *testing.T) {
    c := Choice{}
    require.Empty(t, c.Label); require.Empty(t, c.Value);
    require.Empty(t, c.Preview)
}
```

```

func TestQuestion_Validate_OK(t *testing.T) {
    q := Question{
        Question: "Pick one",
        Choices: []Choice{{Label: "A", Value: "a"}, {Label:
            "B", Value: "b"}},
    }
    require.NoError(t, q.Validate())
}

func TestQuestion_Validate_OK_WithDefault(t *testing.T) {
    q := Question{
        Question: "Pick one",
        Choices: []Choice{{Label: "A", Value: "a"}, {Label:
            "B", Value: "b"}},
        Default: "b",
    }
    require.NoError(t, q.Validate())
}

func TestQuestion_Validate_EmptyText_Err(t *testing.T) {
    q := Question{Choices: []Choice{{Label: "A", Value: "a"},
        {Label: "B", Value: "b"}}}
    require.ErrorIs(t, q.Validate(), ErrInvalidQuestion)
}

func TestQuestion_Validate_TooFewChoices_Err(t *testing.T) {
    q := Question{Question: "x", Choices: []Choice{{Label: "A",
        Value: "a"}}}
    require.ErrorIs(t, q.Validate(), ErrInvalidQuestion)
}

func TestQuestion_Validate_EmptyLabel_Err(t *testing.T) {
    q := Question{Question: "x", Choices: []Choice{{Value: "a"},
        {Label: "B", Value: "b"}}}
    require.ErrorIs(t, q.Validate(), ErrInvalidQuestion)
}

func TestQuestion_Validate_EmptyValue_Err(t *testing.T) {
    q := Question{Question: "x", Choices: []Choice{{Label: "A",
        {Label: "B", Value: "b"}}}
    require.ErrorIs(t, q.Validate(), ErrInvalidQuestion)
}

func TestQuestion_Validate_DuplicateValue_Err(t *testing.T) {
    q := Question{Question: "x", Choices: []Choice{{Label: "A",
        Value: "a"}, {Label: "B", Value: "a"}}}
    require.ErrorIs(t, q.Validate(), ErrInvalidQuestion)
}

```

```

}

func TestQuestion_Validate_DefaultNotInChoices_Err(t *testing.T)
{
    q := Question{
        Question: "x",
        Choices: []Choice{{Label: "A", Value: "a"}, {Label:
            "B", Value: "b"}},
        Default: "z",
    }
    require.ErrorIs(t, q.Validate(), ErrInvalidQuestion)
}

func TestErrorSentinels_DistinctErrorsIs(t *testing.T) {
    for _, e := range []error{
        ErrInvalidQuestion, ErrNoTTYNoDefault, ErrUserCancelled,
        ErrPromptTimeout, ErrTooManyInvalidAttempts,
    } {
        wrapped := fmt.Errorf("wrapped: %w", e)
        require.ErrorIs(t, wrapped, e)
    }
}

func TestConstants_DefaultTimeoutAndRetries(t *testing.T) {
    require.Equal(t, 5*time.Minute, DefaultTimeout)
    require.Equal(t, 3, DefaultMaxRetries)
    require.Equal(t, "stdin", SourceStdin)
    require.Equal(t, "default", SourceDefault)
}

```

Subject: feat(P1-F19-T02): askuser types - Choice + Question + Result + Prompter interface + sentinels.

Task 3: stdin_prompter.go (TDD)

Files: new helix_code/internal/tools/askuser/stdin_prompter.go, new helix_code/internal/tools/askuser/stdin_prompter_test.go.

stdin_prompter.go:

```

type StdinPrompterOptions struct {
    Reader      io.Reader
    Writer      io.Writer
    IsTTY       func() bool
    Renderer    render.Renderer
    Timeout     time.Duration
    MaxRetries  int
}

```

```

type stdinPrompter struct {
    reader      io.Reader
    writer      io.Writer
    isTTY       func() bool
    renderer    render.Renderer
    timeout     time.Duration
    maxRetries  int
}

func NewStdinPrompter(opts StdinPrompterOptions) *stdinPrompter {
    p := &stdinPrompter{
        reader:      opts.Reader,
        writer:      opts.Writer,
        isTTY:       opts.IsTTY,
        renderer:    opts.Renderer,
        timeout:     opts.Timeout,
        maxRetries:  opts.MaxRetries,
    }
    if p.reader == nil { p.reader = os.Stdin }
    if p.writer == nil { p.writer = os.Stdout }
    if p.isTTY == nil { p.isTTY = func() bool { return
        term.IsTerminal(int(os.Stdin.Fd())) } }
    if p.renderer == nil { r, _, _ :=
        render.NewRenderer(render.FactoryOptions{Writer:
            p.writer}); p.renderer = r }
    if p.timeout == 0 { p.timeout = DefaultTimeout }
    if p.maxRetries == 0 { p.maxRetries = DefaultMaxRetries }
    return p
}

// Prompt – see spec §4.2 (non-TTY) + §4.3 (TTY)
func (p *stdinPrompter) Prompt(ctx context.Context, q Question)
    (Result, error) { ... }

// FormatQuestion is exported (spec §11 #5). Pure function: given
// a Question,
// returns the lines that make up the rendered prompt menu in
// display order.
func FormatQuestion(q Question) []string { ... }

```

Implementation notes (spec §4.2 + §4.3 + §5.2): - FormatQuestion: emit q.Question line; blank line; for each choice i: if Preview != "", split on \n and emit each line; emit fmt.Sprintf("%d. %s", i+1, choice.Label); final blank line. - Non-TTY branch: validate q first (so caller gets ErrInvalidQuestion even on non-TTY); if !isTTY() and default is set → linear scan to find matching choice; if !isTTY() and no default → ErrNoTTYNoDefault. **Zero reads from p.reader.** - TTY branch: render menu via render.RenderLines(p.renderer, "ask-user", FormatQuestion(q)); emit prompt text via fmt.Fprint(p.writer, ...); spawn goroutine doing

bufio.NewReader(p.reader).ReadString('\n'); select on ctx.Done() vs result chan; classify input → number / empty / EOF / out-of-range / non-numeric. - Retry loop: for attempt := 0; attempt < p.maxRetries; attempt++. Each iteration emits the prompt line again, reads a fresh line from the buffered reader. Counter resets on each Prompt call. - EOF → ErrUserCancelled. Empty input + default → return default with Source="default". Empty + no default → re-prompt with "Empty input; ..." message; counts toward retries. Non-numeric / out-of-range → re-prompt with "Invalid choice; ..." message; counts toward retries.

Failing tests FIRST (real bytes.Buffer for I/O; injected IsTTY closure):

```
func newPrompter(reader io.Reader, writer io.Writer, isTTY bool)
    *stdinPrompter {
    return NewStdinPrompter(StdinPrompterOptions{
        Reader: reader, Writer: writer,
        IsTTY: func() bool { return isTTY },
        Timeout: 30 * time.Second,
        MaxRetries: 3,
    })
}

func sampleQuestion() Question {
    return Question{
        Question: "Pick one",
        Choices: []Choice{
            {Label: "Option A", Value: "opt-a", Preview:
                "preview-1"},
            {Label: "Option B", Value: "opt-b", Preview:
                "preview-2"},
        },
    }
}

func TestFormatQuestion_NoPreview_NumberedMenuOnly(t *testing.T)
{
    q := Question{
        Question: "Pick", Choices: []Choice{{Label: "A", Value:
            "a"}, {Label: "B", Value: "b"}},
    }
    lines := FormatQuestion(q)
    require.Contains(t, lines, "1. A")
    require.Contains(t, lines, "2. B")
    for _, l := range lines { require.NotContains(t, l,
        "preview") }
}

func TestFormatQuestion_PreviewBeforeLabel_ByteOrder(t
    *testing.T) {
    q := sampleQuestion()
    lines := FormatQuestion(q)
```

```

joined := strings.Join(lines, "\n")
p1 := strings.Index(joined, "preview-1")
l1 := strings.Index(joined, "1. Option A")
p2 := strings.Index(joined, "preview-2")
l2 := strings.Index(joined, "2. Option B")
require.GreaterOrEqual(t, p1, 0)
require.Less(t, p1, l1, "preview-1 must precede label 1")
require.GreaterOrEqual(t, p2, 0)
require.Less(t, p2, l2, "preview-2 must precede label 2")
}

func TestFormatQuestion_MultiLinePreview_LinesPreserved(t
    *testing.T) {
    q := Question{
        Question: "x",
        Choices: []Choice{
            {Label: "A", Value: "a", Preview: "line1\nline2"},
            {Label: "B", Value: "b"},
        },
    }
    lines := FormatQuestion(q)
    require.Contains(t, lines, "line1")
    require.Contains(t, lines, "line2")
}

func TestStdinPrompter_NonTTY_NoDefault_ErrNoTTY_ZeroReads(t
    *testing.T) {
    reader := bytes.NewBufferString("NEVER\n")
    var writer bytes.Buffer
    p := newPrompter(reader, &writer, false)
    _, err := p.Prompt(context.Background(), sampleQuestion())
    require.ErrorIs(t, err, ErrNoTTYNoDefault)
    require.Contains(t, err.Error(), "interactive terminal")
    require.Equal(t, "NEVER\n", reader.String(), "must NOT
        consume any bytes")
}

func TestStdinPrompter_NonTTY_WithDefault_ReturnsDefault_ZeroReads(t
    *testing.T) {
    reader := bytes.NewBufferString("NEVER\n")
    var writer bytes.Buffer
    p := newPrompter(reader, &writer, false)
    q := sampleQuestion(); q.Default = "opt-b"
    res, err := p.Prompt(context.Background(), q)
    require.NoError(t, err)
    require.Equal(t, "opt-b", res.Value)
    require.Equal(t, 1, res.Index)
    require.Equal(t, SourceDefault, res.Source)
}

```



```

        require.Equal(t, "NEVER\n", reader.String(), "must NOT
            consume any bytes")
    }

func TestStdinPrompter_TTY_ValidInput_ReturnsChoice(t
    *testing.T) {
    reader := bytes.NewBufferString("2\n")
    var writer bytes.Buffer
    p := newPrompter(reader, &writer, true)
    res, err := p.Prompt(context.Background(), sampleQuestion())
    require.NoError(t, err)
    require.Equal(t, "opt-b", res.Value)
    require.Equal(t, 1, res.Index)
    require.Equal(t, SourceStdin, res.Source)
}

func TestStdinPrompter_TTY_EmptyWithDefault_ReturnsDefault(t
    *testing.T) {
    reader := bytes.NewBufferString("\n")
    var writer bytes.Buffer
    p := newPrompter(reader, &writer, true)
    q := sampleQuestion(); q.Default = "opt-a"
    res, err := p.Prompt(context.Background(), q)
    require.NoError(t, err)
    require.Equal(t, SourceDefault, res.Source)
    require.Equal(t, "opt-a", res.Value)
}

func TestStdinPrompter_TTY_OutOfRange_RePromptsThenAccepts(t
    *testing.T) {
    reader := bytes.NewBufferString("9\n1\n")
    var writer bytes.Buffer
    p := newPrompter(reader, &writer, true)
    res, err := p.Prompt(context.Background(), sampleQuestion())
    require.NoError(t, err)
    require.Equal(t, 0, res.Index)
    require.Equal(t, 1, strings.Count(writer.String(), "Invalid
        choice"))
}

func TestStdinPrompter_TTY_NonNumeric_RePromptsThenAccepts(t
    *testing.T) {
    reader := bytes.NewBufferString("abc\n2\n")
    var writer bytes.Buffer
    p := newPrompter(reader, &writer, true)
    res, err := p.Prompt(context.Background(), sampleQuestion())
    require.NoError(t, err)
    require.Equal(t, 1, res.Index)
}

```

```

        require.Contains(t, writer.String(), "Invalid choice")
    }

func TestStdinPrompter_TTY_ThreeInvalids_ErrTooMany(t
    *testing.T) {
    reader := bytes.NewBufferString("a\nb\nc\n")
    var writer bytes.Buffer
    p := newPrompter(reader, &writer, true)
    _, err := p.Prompt(context.Background(), sampleQuestion())
    require.ErrorIs(t, err, ErrTooManyInvalidAttempts)
}

func TestStdinPrompter_TTY_EOF_ErrUserCancelled(t *testing.T) {
    reader := bytes.NewBufferString("") // empty → EOF on first
        read
    var writer bytes.Buffer
    p := newPrompter(reader, &writer, true)
    _, err := p.Prompt(context.Background(), sampleQuestion())
    require.ErrorIs(t, err, ErrUserCancelled)
}

func TestStdinPrompter_TTY_CtxCancel_ErrPromptTimeout(t
    *testing.T) {
    pr, _ := io.Pipe() // never written → blocks
    var writer bytes.Buffer
    p := NewStdinPrompter(StdinPrompterOptions{
        Reader: pr, Writer: &writer,
        IsTTY: func() bool { return true },
        Timeout: 30 * time.Second, MaxRetries: 3,
    })
    ctx, cancel := context.WithCancel(context.Background())
    go func() { time.Sleep(50 * time.Millisecond); cancel() }()
    _, err := p.Prompt(ctx, sampleQuestion())
    require.ErrorIs(t, err, ErrPromptTimeout)
}

func TestStdinPrompter_TTY_TimeoutExpires_ErrPromptTimeout(t
    *testing.T) {
    pr, _ := io.Pipe()
    var writer bytes.Buffer
    p := NewStdinPrompter(StdinPrompterOptions{
        Reader: pr, Writer: &writer,
        IsTTY: func() bool { return true },
        Timeout: 50 * time.Millisecond, MaxRetries: 3,
    })
    _, err := p.Prompt(context.Background(), sampleQuestion())
    require.ErrorIs(t, err, ErrPromptTimeout)
}

```

```

func TestStdinPrompter_TTY_PreviewRenderedBeforeLabel_ByteOffset(t
    *testing.T) {
    reader := bytes.NewBufferString("1\n")
    var writer bytes.Buffer
    p := newPrompter(reader, &writer, true)
    _, err := p.Prompt(context.Background(), sampleQuestion())
    require.NoError(t, err)
    cap := writer.Bytes()
    p1 := bytes.Index(cap, []byte("preview-1"))
    l1 := bytes.Index(cap, []byte("1. Option A"))
    p2 := bytes.Index(cap, []byte("preview-2"))
    l2 := bytes.Index(cap, []byte("2. Option B"))
    require.GreaterOrEqual(t, p1, 0); require.Less(t, p1, l1)
    require.GreaterOrEqual(t, p2, 0); require.Less(t, p2, l2)
}

```

```

func TestStdinPrompter_RetryCounter_ResetsAcrossPromptCalls(t
    *testing.T) {
    // Two consecutive prompts on the SAME reader: first has 2
    // invalids + valid;
    // second has 2 invalids + valid. Both must succeed (counter
    // resets).
    reader := bytes.NewBufferString("a\nb\n1\nc\nd\n2\n")
    var writer bytes.Buffer
    p := newPrompter(reader, &writer, true)
    r1, err := p.Prompt(context.Background(), sampleQuestion());
    require.NoError(t, err)
    require.Equal(t, 0, r1.Index)
    r2, err := p.Prompt(context.Background(), sampleQuestion());
    require.NoError(t, err)
    require.Equal(t, 1, r2.Index)
}

```

```

func TestStdinPrompter_TTY_DefaultDoesNotPreemptInput(t
    *testing.T) {
    // IsTTY=true + default set + reader has "1\n" → returns
    // Index 0 (the user's
    // explicit "1"), NOT the default. Pins the anti-bluff branch
    // where Prompt
    // short-circuits to default regardless of IsTTY.
    reader := bytes.NewBufferString("1\n")
    var writer bytes.Buffer
    p := newPrompter(reader, &writer, true)
    q := sampleQuestion(); q.Default = "opt-b"
    res, err := p.Prompt(context.Background(), q)
    require.NoError(t, err)
    require.Equal(t, 0, res.Index)
}

```

```

    require.Equal(t, SourceStdin, res.Source)
}

func TestNoLoggerInfoOrDebugTakesQuestionOrInput_CONST042_SourceScan(t
    *testing.T) {
    matches, err := filepath.Glob("*.go")
    require.NoError(t, err)
    forbidden := regexp.MustCompile(`logger\\.\\b(Info|Debug)
        \\b.*\\b(question|preview|label|input|answer)\\b`)
    for _, f := range matches {
        b, err := os.ReadFile(f); require.NoError(t, err)
        require.False(t, forbidden.Match(b), "%s contains
            forbidden logger call: CONST-042", f)
    }
}

```

Subject: feat(P1-F19-T03): stdinPrompter with non-TTY short-circuit
+ retry loop + timeout + F18 menu render.

Task 4: ask_user_tool.go (TDD) + registry category

Files: new helix_code/internal/tools/askuser/ask_user_tool.go, new
helix_code/internal/tools/askuser/ask_user_tool_test.go, modify
helix_code/internal/tools/registry.go (add CategoryAskUser const).

ask_user_tool.go:

```

type AskUserTool struct {
    prompter Prompter
}

func NewAskUserTool(p Prompter) *AskUserTool { return
    &AskUserTool{prompter: p} }

func (t *AskUserTool) Name() string { return "ask_user" }

func (t *AskUserTool) Description() string {
    return
        "Ask the user a multiple-choice question with optional
        inline previews. " +
        "Returns the user's chosen value, label, 0-based index,
        and source ('stdin' or 'default'). " +

        "On non-interactive stdin (CI / pipe) and a default value
        is provided, returns the default; " +
        "if no default is provided, returns an error."
}

```

```

func (t *AskUserTool) Category() tools.ToolCategory { return
    tools.CategoryAskUser }

func (t *AskUserTool) Schema() tools.ToolSchema {
    return tools.ToolSchema{
        Type: "object",
        Properties: map[string]interface{}{
            "question": map[string]interface{}{"type": "string",
            "description": "The question text to show the user.
            Required."},
            "choices": map[string]interface{}{
                "type": "array",
                "description": "Ordered list of choices. Each
                choice has label (required), value (required), preview
                (optional inline context shown above the label). Minimum
                2 choices.",
                "items": map[string]interface{}{
                    "type": "object",
                    "properties": map[string]interface{}{
                        "label": map[string]interface{}
{"type": "string"},
                        "value": map[string]interface{}
{"type": "string"},
                        "preview": map[string]interface{}
{"type": "string"},
                    },
                    "required": []string{"label", "value"},
                },
                "minItems": 2,
            },
            "default": map[string]interface{}{"type": "string",
            "description":
            "Optional. If set, MUST equal one choice's value. Picked
            when stdin is non-TTY or user presses Enter without
            typing a number."},
        },
        Required: []string{"question", "choices"},
        Description: "Pause the agent loop and ask the user to
        pick one of N choices.",
    }
}

func (t *AskUserTool) Validate(params map[string]interface{})
    error {
    q, err := parseParams(params)
    if err != nil { return err }
    return q.Validate()
}

```

```

func (t *AskUserTool) Execute(ctx context.Context, params
    map[string]interface{}) (interface{}, error) {
    if t.prompter == nil { return nil, fmt.Errorf("ask_user:
        prompter not configured") }
    q, err := parseParams(params)
    if err != nil { return nil, err }
    if err := q.Validate(); err != nil { return nil, err }
    res, err := t.prompter.Prompt(ctx, q)
    if err != nil { return nil, err }
    return map[string]interface{}{
        "value": res.Value,
        "label": res.Label,
        "index": res.Index,
        "source": res.Source,
    }, nil
}

```

```

// parseParams converts map[string]interface{} → Question,
// surfacing type errors clearly.
func parseParams(params map[string]interface{}) (Question,
    error) { ... }

```

registry.go change (one line in const block):

```

const (
    // existing categories...
    CategoryAskUser ToolCategory = "ask-user"
)

```

(The actual registration lives in T05 to keep T04 focused on the tool unit.)

Failing tests FIRST (ask_user_tool_test.go):

```

type fakePrompter struct {
    calls []Question
    result Result
    err    error
}
func (f *fakePrompter) Prompt(ctx context.Context, q Question)
    (Result, error) {
    f.calls = append(f.calls, q); return f.result, f.err
}

```

```

func validParams() map[string]interface{} {
    return map[string]interface{}{
        "question": "Pick",
        "choices": []interface{}{
            map[string]interface{}{"label": "A", "value": "a"},
            map[string]interface{}{"label": "B", "value": "b"},
        },
    }
}

```

```

    },
}
}

func TestAskUserTool_Name_IsAskUser(t *testing.T) {
    require.Equal(t, "ask_user",
        NewAskUserTool(&fakePrompter{}).Name())
}

func TestAskUserTool_Category_IsAskUser(t *testing.T) {
    require.Equal(t, tools.CategoryAskUser,
        NewAskUserTool(&fakePrompter{}).Category())
}

func TestAskUserTool_Description_MentionsChoices(t *testing.T) {
    d := NewAskUserTool(&fakePrompter{}).Description()
    require.Contains(t, d, "choice")
    require.Contains(t, d, "default")
}

func TestAskUserTool_Schema_HasRequiredFields(t *testing.T) {
    s := NewAskUserTool(&fakePrompter{}).Schema()
    require.ElementsMatch(t, []string{"question", "choices"},
        s.Required)
}

func TestAskUserTool_Validate_OK(t *testing.T) {
    require.NoError(t,
        NewAskUserTool(&fakePrompter{}).Validate(validParams()))
}

func TestAskUserTool_Validate_MissingQuestion_Err(t *testing.T) {
    p := validParams(); delete(p, "question")
    err := NewAskUserTool(&fakePrompter{}).Validate(p)
    require.Error(t, err)
}

func TestAskUserTool_Validate_ChoicesWrongType_Err(t *testing.T) {
    {
        p := validParams(); p["choices"] = "not an array"
        err := NewAskUserTool(&fakePrompter{}).Validate(p)
        require.Error(t, err)
    }
}

func TestAskUserTool_Execute_DispatchesToPrompter(t *testing.T) {
    fp := &fakePrompter{result: Result{Value: "b", Label: "B",
        Index: 1, Source: SourceStdin}}
    out, err := NewAskUserTool(fp).Execute(context.Background(),
        validParams())
    require.NoError(t, err)
    require.Len(t, fp.calls, 1)
    require.Equal(t, "Pick", fp.calls[0].Question)
}

func TestAskUserTool_Execute_ReturnsMapShape(t *testing.T) {

```

```

    fp := &fakePrompter{result: Result{Value: "b", Label: "B",
        Index: 1, Source: SourceStdin}}
    out, err := NewAskUserTool(fp).Execute(context.Background(),
        validParams())
    require.NoError(t, err)
    m, ok := out.(map[string]interface{}); require.True(t, ok)
    require.Equal(t, "b", m["value"])
    require.Equal(t, "B", m["label"])
    require.Equal(t, 1, m["index"])
    require.Equal(t, SourceStdin, m["source"])
}
func TestAskUserTool_Execute_PrompterError_Propagated(t
    *testing.T) {
    fp := &fakePrompter{err: ErrUserCancelled}
    _, err := NewAskUserTool(fp).Execute(context.Background(),
        validParams())
    require.ErrorIs(t, err, ErrUserCancelled)
}
func TestAskUserTool_Execute_NilPrompter_Err(t *testing.T) {
    _, err := NewAskUserTool(nil).Execute(context.Background(),
        validParams())
    require.Error(t, err)
    require.Contains(t, err.Error(), "ask_user")
}

```

Subject: feat(P1-F19-T04): AskUserTool wrapping Prompter + CategoryAskUser registry const.

Task 5: registry wiring + integration test

Files: modify `helix_code/internal/tools/registry.go` (register `askuser.NewAskUserTool` in `buildToolList` or equivalent + add optional `AskUserPrompter askuser.Prompter` field on `RegistryConfig`), modify `helix_code/cmd/cli/main.go` (optional: pass `c.renderer` via `RegistryConfig.AskUserPrompter`), new `helix_code/tests/integration/askuser_test.go`.

`registry.go` changes:

1. Add field `AskUserPrompter askuser.Prompter` to `RegistryConfig`.
2. In the registration block, add:

```

var askUserPrompter askuser.Prompter
if cfg.AskUserPrompter != nil {
    askUserPrompter = cfg.AskUserPrompter
} else {
    askUserPrompter =
        askuser.NewStdinPrompter(askuser.StdinPrompterOptions{})
}

```



```

}
r.tools["ask_user"] = askuser.NewAskUserTool(askUserPrompter)

```

3. New import: `dev.helix.code/internal/tools/askuser`.

Integration tests (`tests/integration/askuser_test.go` — `//go:build integration`; ALWAYS-runs; no infrastructure dep):

```
//go:build integration
```

```
package integration
```

```

func TestAskUser_Integration_TTY_RealRender_ContainsPreviewBeforeLabel(t
    *testing.T) {
    reader := bytes.NewBufferString("1\n")
    var writer bytes.Buffer
    p := askuser.NewStdinPrompter(askuser.StdinPrompterOptions{
        Reader: reader, Writer: &writer,
        IsTTY: func() bool { return true },
        Timeout: 5 * time.Second, MaxRetries: 3,
    })
    tool := askuser.NewAskUserTool(p)
    out, err := tool.Execute(context.Background(),
        map[string]interface{}{
            "question": "Pick",
            "choices": []interface{}{
                map[string]interface{}{"label": "A", "value": "a",
                    "preview": "preview-A"},
                map[string]interface{}{"label": "B", "value": "b",
                    "preview": "preview-B"},
            },
        })
    require.NoError(t, err)
    m := out.(map[string]interface{})
    require.Equal(t, "a", m["value"])
    cap := writer.Bytes()
    require.Less(t, bytes.Index(cap, []byte("preview-A")),
        bytes.Index(cap, []byte("1. A")))
    require.Less(t, bytes.Index(cap, []byte("preview-B")),
        bytes.Index(cap, []byte("2. B")))
}

```

```

func TestAskUser_Integration_NonTTY_Default_ReturnsDefault_ZeroReads(t
    *testing.T) {
    reader := bytes.NewBufferString("NEVER\n")
    var writer bytes.Buffer
    p := askuser.NewStdinPrompter(askuser.StdinPrompterOptions{
        Reader: reader, Writer: &writer,
        IsTTY: func() bool { return false },
    })
}

```

```

        Timeout: 5 * time.Second, MaxRetries: 3,
    ))
    tool := askuser.NewAskUserTool(p)
    out, err := tool.Execute(context.Background(),
        map[string]interface{}{
            "question": "Pick",
            "choices": []interface{}{
                map[string]interface{}{"label": "A", "value": "a"},
                map[string]interface{}{"label": "B", "value": "b"},
            },
            "default": "b",
        })
    require.NoError(t, err)
    m := out.(map[string]interface{})
    require.Equal(t, "b", m["value"])
    require.Equal(t, askuser.SourceDefault, m["source"])
    require.Equal(t, "NEVER\n", reader.String(), "must NOT
        consume any bytes")
}

```

```

func TestAskUser_Integration_NonTTY_NoDefault_ErrPropagated(t
    *testing.T) {
    reader := bytes.NewBufferString("NEVER\n")
    var writer bytes.Buffer
    p := askuser.NewStdinPrompter(askuser.StdinPrompterOptions{
        Reader: reader, Writer: &writer,
        IsTTY: func() bool { return false },
        Timeout: 5 * time.Second, MaxRetries: 3,
    })
    tool := askuser.NewAskUserTool(p)
    _, err := tool.Execute(context.Background(),
        map[string]interface{}{
            "question": "Pick",
            "choices": []interface{}{
                map[string]interface{}{"label": "A", "value": "a"},
                map[string]interface{}{"label": "B", "value": "b"},
            },
        })
    require.ErrorIs(t, err, askuser.ErrNoTTYNoDefault)
    require.Contains(t, err.Error(), "interactive terminal")
    require.Equal(t, "NEVER\n", reader.String())
}

```

```

func TestAskUser_Integration_TwoConsecutivePrompts_ReaderConsumed(t
    *testing.T) {
    reader := bytes.NewBufferString("1\n2\n")
    var writer bytes.Buffer
    p := askuser.NewStdinPrompter(askuser.StdinPrompterOptions{

```

```

    Reader: reader, Writer: &writer,
    IsTTY: func() bool { return true },
    Timeout: 5 * time.Second, MaxRetries: 3,
  })
  tool := askuser.NewAskUserTool(p)
  params := map[string]interface{}{
    "question": "Pick",
    "choices": []interface{}{
      map[string]interface{}{"label": "A", "value": "a"},
      map[string]interface{}{"label": "B", "value": "b"},
    },
  }
  out1, err := tool.Execute(context.Background(), params);
  require.NoError(t, err)
  require.Equal(t, 0, out1.(map[string]interface{})["index"])
  out2, err := tool.Execute(context.Background(), params);
  require.NoError(t, err)
  require.Equal(t, 1, out2.(map[string]interface{})["index"])
}

```

Subject: feat(P1-F19-T05): wire ask_user into registry + integration test (always-runs both branches).

Task 6: Challenge harness (5-phase, positive evidence)

Files: new helix_code/tests/integration/cmd/p1f19_challenge/main.go, new challenges/p1-f19-ask-user-question/CHALLENGE.md, new challenges/p1-f19-ask-user-question/run.sh.

Harness phases (per spec §6.3):

1. **PHASE-A: STDIN-INPUT (always runs)** — TTY=true; reader="2\n"; assert result["value"]=="opt-b", result["index"]==1, result["source"]=="stdin".
2. **PHASE-B: NON-TTY-WITH-DEFAULT (always runs)** — TTY=false; default="opt-b"; reader="NEVER\n"; assert result["source"]=="default", result["value"]=="opt-b", reader's bytes unchanged.
3. **PHASE-C: NON-TTY-WITHOUT-DEFAULT-ERR (always runs)** — TTY=false; no default; reader="NEVER\n"; assert errors.Is(err, ErrNoTTYNoDefault), error text contains "interactive terminal", reader's bytes unchanged.
4. **PHASE-D: PREVIEW-RENDERING (always runs)** — TTY=true; 2 choices each with preview; reader="1\n"; assert captured writer contains both preview strings; assert byte offsets put each preview BEFORE its choice label.
5. **PHASE-E: INVALID-INPUT-RETRY (always runs)** — TTY=true; reader="9\n1\n"; assert result["index"]==0, captured writer contains "Invalid choice" exactly once. Sub-case: reader="a\nb\nc\n"; assert errors.Is(err, ErrTooManyInvalidAttempts).

Output skeleton (verbatim per spec §6.3) ends with:

SUMMARY: PHASE-A=7/7 PASS; PHASE-B=5/5 PASS; PHASE-C=4/4 PASS; PHASE-D=5/5

The Challenge MUST exit non-zero on any byte-evidence mismatch. Absence-of-error is NEVER acceptable. Reader-position invariants (Phases B and C) and byte-offset invariants (Phase D) are positive evidence of real stdin/render flow. Anti-bluff smoke clean check appended to harness output. Verbatim output captured into `06_phase_1_evidence.md`. Dual commit (Challenges submodule + meta-repo bump).

Subject: feat(P1-F19-T06): challenge with 5 always-run phases + reader-position + byte-offset positive evidence.

Task 7: Close-out + push

Tick all 7 items in PROGRESS, advance PROGRESS focus to F20 candidate (Theme System per porting doc), run final verification:

```
cd HelixCode && make verify-compile
grep -rn "simulated\|for now\|TODO implement\|placeholder" \
  internal/tools/askuser && echo BLUFF || echo clean
go test -count=1 ./internal/tools/askuser/...
go test -count=1 -tags=integration ./tests/integration/...
go mod tidy
# go.mod expected change: NONE
# go.sum expected change: NONE
git diff --exit-code go.mod go.sum # MUST be no-op
```

Commit chore(P1-F19-T07): close out feature 19 — ask user question with previews. Push 4 remotes non-force (origin, helixdev, vasic-digital, gitlab per programme conventions). Request explicit user authorization at this step (CONST-043).

Self-review notes

1. **Spec coverage:** every spec section maps to a task — T02 types + interface (§3.3), T03 `stdinPrompter` (§4.2 + §4.3 + §5.2 (a)–(d) anti-bluff defences), T04 `AskUserTool` (§3.3) + registry category `const`, T05 registry wire + integration tests (§6.2), T06 Challenge five phases (§5.2 + §6.3), T07 close-out (§9).
2. **TDD:** every code task starts with failing tests. `Prompter` impl tests against real `bytes.Buffer` reader/writer + injected `IsTTY` closure (real `io.Reader/io.Writer` seams, no mocks of `stdlib`). Tool tests use `fakePrompter` (the prompter is the dependency-under-mock, NOT the system-under-test). Integration tests wire up the actual `AskUserTool` + `stdinPrompter` (production class) via the same `bytes.Buffer` seams to prove the production hot path works in non-TTY mode.
3. **Type consistency:** `Choice`, `Question`, `Result`, `Prompter`, `StdinPrompterOptions`, error sentinels (`ErrInvalidQuestion`, `ErrNoTTYNoDefault`, `ErrUserCancelled`, `ErrPromptTimeout`, `ErrTooManyInvalidAttempts`), constants (`DefaultTimeout`, `DefaultMaxRetries`, `SourceStdin`, `SourceDefault`), tool name `ask_user`, category `tools.CategoryAskUser` — all match across spec §3.3 and plan T02–T05.

4. **Zero new external deps:** `stdlib` + existing `testify/zap` + `golang.org/x/term` (already direct dep after F18) + F18-internal `internal/render.go` `mod tidy` after T03 produces NO changes to `go.mod` AND NO new entries in `go.sum`. T07's verification step asserts this loudly.
5. **Anti-bluff (§5.2):** Challenge has FIVE always-run phases. Every phase records positive evidence (Result fields, captured byte content, byte offsets, reader-position invariants). The four real-execution criteria — (a) Prompt actually reads from injected reader (Phase A asserts `Result.Index` reflects the reader's first line); (b) non-TTY error path tested (Phase C asserts `ErrNoTTYNoDefault` + reader bytes unchanged); (c) production class exercised (Phases B/C/D/E all use real `stdinPrompter`, never a mock); (d) preview text appears BEFORE label (Phase D asserts byte-offset ordering) — each have dedicated unit + integration + Challenge assertions. Byte-evidence mismatch is a hard Challenge failure.
6. **CONST-042:** the prompter NEVER logs question text, preview text, label text, or user input at any level. Source-scan test (`TestNoLoggerInfoOrDebugTakesQuestionOrInput_CONST042_SourceScan` in T03) enforces this with a regex. The Challenge's saved evidence file records `Result.Value` (stable identifier from the agent), byte counts, and byte-offset relationships — never the rendered prompt text or user's typed line.
7. **CONST-043:** stays on `main`, non-force to all four remotes; explicit user authorization is requested at T07 before pushing.
8. **Stdin readline vs library (Q1=A) — non-obvious call** (recorded in spec §2 trailer): no new deps + small render surface + `applications/terminal_ui/` already uses `tview/tcell` for the full TUI; adding `bubbletea/promptui` here would be a third paradigm. The prompter is ~150 lines of pure Go; a library would be 30 KLOC.
9. **Tool only, no slash, no cobra (Q5=A) — non-obvious call:** `ask_user` is agent-driven (the LLM emits a `tool_use` block); a `/ask slash` inverts the semantics (user types the question, not the agent). A cobra subcommand is debug-only convenience deferred to F19.5.
10. **5-minute timeout default — non-obvious call** (recorded in spec §11 #1): biases toward “agent reports the timeout sooner so the user can re-issue the prompt” vs `claude-code`'s longer wait. Configurable via `StdinPrompterOptions.Timeout`.
11. **3 retries before ErrTooManyInvalidAttempts — non-obvious call** (recorded in spec §11 #2): caps CI infinite-loop risk vs typo forgiveness. Configurable via `StdinPrompterOptions.MaxRetries`.
12. **EOF returns ErrUserCancelled (NOT silent default) — non-obvious call** (recorded in spec §11 #3): explicit cancellation signal; silent default would mask piped-stdin misconfigurations. Distinct from non-TTY-with-default short-circuit (which fires BEFORE any read on the basis of `IsTTY()=false`).
13. **FormatQuestion exported as a pure helper — non-obvious call** (recorded in spec §11 #5): lets the anti-bluff (d) test pin byte order WITHOUT going through `Prompt`; reusable from a future `/ask slash` command (F19.5).
14. **Single preview string vs typed previews — non-obvious call** (recorded in spec §11 #6): `claude-code`'s `QuestionPreview{type, content, title}` is collapsed to a single string in v1; typed dispatch deferred to F19.5 (would couple F19 to a markdown library / image protocol).
15. **Tool returns map[string]interface{} not typed Result — non-obvious call** (recorded in spec §11 #7): matches existing F14/F15/F17 tool return convention; LLM-side JSON decoder sees a stable shape.
16. **Goroutine-leak on cancellation acceptable v1 — non-obvious call** (recorded in spec §11 #10): `bufio.ReadString('\n')` blocks on the kernel; `ctx-cancel` returns from the prompter but the read goroutine stays parked until the next byte. v2 may use `os.Stdin.SetReadDeadline`. Documented.